tf.vectorized_map Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/vectorized_map

> *Parallel map on the list of tensors unpacked from elems on dimension 0.*

Function Signatures

```
tf.vectorized_map( fn, elems, fallback_to_while_loop=True,
warn=True )
```

Code Examples

```
tf.vectorized_map(
fn, elems, fallback_to_while_loop=True, warn=True
)
```


```
tf.vectorized_map(
fn, elems, fallback_to_while_loop=True, warn=True
)
```

```
def outer_product(a):
return tf.tensordot(a, a, 0)
batch_size = 100
a = tf.ones((batch_size, 32, 32))
c = tf.vectorized_map(outer_product, a)
assert c.shape == (batch_size, 32, 32, 32, 32)
```


```
def outer_product(a):
return tf.tensordot(a, a, 0)
batch_size = 100
a = tf.ones((batch_size, 32, 32))
c = tf.vectorized_map(outer_product, a)
assert c.shape == (batch_size, 32, 32, 32, 32)
```


```
# Computing per-example gradients
batch_size = 10
num_features = 32
layer = tf.keras.layers.Dense(1)
def model_fn(arg):
with tf.GradientTape() as g:
inp, label = arg
inp = tf.expand_dims(inp, 0)
label = tf.expand_dims(label, 0)
prediction = layer(inp)
loss = tf.nn.l2_loss(label - prediction)
return g.gradient(loss, (layer.kernel, layer.bias))
inputs = tf.random.uniform([batch_size, num_features])
labels = tf.random.uniform([batch_size, 1])
per_example_gradients = tf.vectorized_map(model_fn, (inputs,
labels))
assert per_example_gradients[0].shape == (batch_size, num_features,
1)
assert per_example_gradients[1].shape == (batch_size, 1)
```

```
# Computing per-example gradients
batch_size = 10
num_features = 32
layer = tf.keras.layers.Dense(1)
def model_fn(arg):
with tf.GradientTape() as g:
inp, label = arg
inp = tf.expand_dims(inp, 0)
label = tf.expand_dims(label, 0)
prediction = layer(inp)
loss = tf.nn.l2_loss(label - prediction)
return g.gradient(loss, (layer.kernel, layer.bias))
inputs = tf.random.uniform([batch_size, num_features])
labels = tf.random.uniform([batch_size, 1])
per_example_gradients = tf.vectorized_map(model_fn, (inputs,
labels))
assert per_example_gradients[0].shape == (batch_size, num_features,
1)
assert per_example_gradients[1].shape == (batch_size, 1)
```

Documentation

Parallel map on the list of tensors unpacked from elems on dimension 0.

View aliases

Compat aliases for migration

See
Migration guide for
more details.

tf.compat.v1.vectorized_map

Used in the notebooks

• NumPy API on TensorFlow

This method works similar to tf.map_fn but is optimized to run much faster, possibly with a much larger memory footprint. The speedups are obtained by vectorization (see Auto-Vectorizing TensorFlow Graphs: Jacobians, Auto-Batching and Beyond). The idea
behind vectorization is to semantically launch all the invocations of fn in parallel and fuse corresponding operations across all these invocations. This fusion is done statically at graph generation time and the generated code is often similar in performance to a manually fused version.

Because tf.vectorized_map fully parallelizes the batch, this method will generally be significantly faster than using tf.map_fn, especially in eager mode. However this is an experimental feature and currently has a lot of limitations:

- There should be no data dependency between the different semantic

invocations of fn, i.e. it should be safe to map the elements of the inputs in any order.

- Stateful kernels may mostly not be supported since these often imply a

data dependency. We do support a limited set of such stateful kernels though (like RandomFoo, Variable operations like reads, etc).

- fn has limited support for control flow operations.
- fn should return nested structure of Tensors or Operations. However

if an Operation is returned, it should have zero outputs.

- The shape and dtype of any intermediate or output tensors in the

computation of fn should not depend on the input to fn.

Examples:

Returns

Although they are less common as user-visible inputs and outputs, note that tensors of type tf.variant which represent tensor lists (for example from tf.raw_ops.TensorListFromTensor) are vectorized by stacking the list contents rather than the variant itself, and so the container tensor will have a scalar shape when returned rather than the usual stacked shape. This improves the performance of control flow gradient vectorization.

Raises
